

Real and Complex FFT Implementation using CMSIS

Electronics Project 4 (ET-PM4): CW-Doppler Radar Sensor

Abstract

Today, Fast Fourier Transform (FFT) functions are typically not implemented manually. Instead, developers rely on SDK or library functions that are already optimised for the target microcontroller platform. In this document, the *CMSIS DSP Software Library Version 1.17.0* is used.

The PM4 project employs the *STM32F429I-DISC1* evaluation board, which is built around the STM32F429I microcontroller. This microcontroller is based on an ARM Cortex-M4F CPU core, where the “F” indicates the presence of a floating-point unit (FPU). Consequently, the floating-point FFT implementation from the CMSIS DSP library is used in this work.

Additionally, the document includes a brief section explaining how ADC data must be converted in order to serve as valid time-domain input signals for the FFT.

Table of Content

Abstract.....	1
1. Review of the Fast Fourier Transform (FFT).....	2
2. CMSIS Implementation	3
2.1. FFT Strukt Initialisation	5
2.2. FFT Preparation and Function Call.....	6
2.2.1. Type Cast from ADC Dual Mode Samples.....	6
2.2.2. Optional Zero Padding	6
2.2.3. FFT Function Call	6
2.2.4. Test Signal.....	7
2.3. Some useful CMSIS Functions	7
2.4. Some possible errors.....	8
2.4.1. __FPU_PRESENT	8

2. CMSIS Implementation

The Cortex Microcontroller Software Interface Standard (CMSIS) is a vendor-independent hardware abstraction layer for microcontrollers that are based on Arm Cortex processors. An overview over all available DSP (digital signal processing) library functions can be found in [1], [CMSIS-DSP Software Library](#). To access the provided functionality in your C code, simply include the header file `arm_math.h`.

Under Reference -> Transform Functions there are, amongst other, lists of complex and real FFT library functions for different data types and, additionally, a description at the end of the pages.

The FFT-computation and the FFT-output are typically complex-valued. However, there are so-called *real FFT* functions, which are optimized for a single real-valued input signal. In the PM4 module the ordinary complex FFT is to be preferred, because the CFFT supports complex-valued input signals too. Moreover, the 32 bit floating-point data type F32 should be preferred because the processing of floating-point numbers takes only marginally longer than the processing of integer numbers thanks to the microcontroller's floating point unit [2] and, additionally, because scaling is no longer necessary. For the interested reader, FFT-cycle-counts for various FFT block lengths N and data types are shown on pages 10 and 11 in [3]. In PM4, the CMSIS-DSP-Library function `arm_cfft_f32` can be used, see Figure 2.

The screenshot shows the CMSIS-DSP Version 1.17.0 software library documentation for the `arm_cfft_f32()` function. It includes the function signature, a description of the processing function, a detailed list of parameters with their roles and possible values, a note about the Neon version API, and a code snippet for the Neon version. It also mentions that buffer sizes are described on the 'transform buffers' page.

CMSIS-DSP Version 1.17.0
CMSIS DSP Software Library

arm_cfft_f32()

```
void arm_cfft_f32 ( const arm_cfft_instance_f32 * S,
                  float32_t * p1,
                  uint8_t ifftFlag,
                  uint8_t bitReverseFlag
                  )
```

Processing function for the floating-point complex FFT.

Parameters

[in]	S	points to an instance of the floating-point CFFT structure
[in,out]	p1	points to the complex data buffer. Processing occurs in-place
[in]	ifftFlag	flag that selects transform direction - value = 0: forward transform - value = 1: inverse transform
[in]	bitReverseFlag	flag that enables / disables bit reversal of output - value = 0: disables bit reversal of output - value = 1: enables bit reversal of output

Neon version

The neon version has a different API. The input and output buffers must be different. There is an optional temporary buffer. If the temporary buffer is not used, the input buffer is modified. The bit reverse flag is not more available in Neon version.

```
void arm_cfft_f32(
    const arm_cfft_instance_f32 * S,
    float32_t * pIn,
    float32_t * pOut,
    float32_t * pBuffer,
    uint8_t ifftFlag);
```

Size of buffers according to the target architecture and datatype:
They are described on the page [transform buffers](#).

Figure 2: Documentation of the [CMSIS-DSP-Library function `arm\_cfft\_f32`](#) [1].

`p1` is a pointer to the complex data buffer with $2 \cdot N$ array-elements of data type `float32_t`. Thus, the input of the function is the address (unary operator `&`) of the first element of the array.

At the function call, the array contains the samples of the 2 radar sensor outputs, and after

leaving the function, the array contains the spectrum values. The input samples are overwritten.

The flags can be defined as constants (after the `#include` section)

```
#define IFFT_FLAG 0
#define BIT_REVERSE_FLAG 1
```

since in PM4 only the FFT with output sorted in ascending order is required.

Finally, the struct `arm_cfft_instance_f32` containing the FFT-length, tables for the twiddle-factors and bit reversed addressing etc. must be initialized for the library function `arm_cfft_f32`, see [CMSIS-DSP Software Library](#) under Data Structures and Figure 3.

CMSIS-DSP Version 1.17.0
CMSIS DSP Software Library

arm_cfft_instance_f32 Struct Reference

Instance structure for the floating-point CFFT/CIFFT function. [More...](#)

Data Fields

uint16_t	<code>fftLen</code>
const float32_t*	<code>pTwiddle</code>
const uint16_t*	<code>pBitRevTable</code>
uint16_t	<code>bitRevLength</code>

Description

Instance structure for the floating-point CFFT/CIFFT function.

Examples

[arm_fft_bin_example_f32.c](#).

Figure 3: Instance structure for floating-point CFFT function [1].

2.1. FFT Struct Initialisation

As a first step in the firmware, you must initialize the above-mentioned structure once: `arm_cfft_instance_f32` for the complex valued input or `arm_rfft_fast_instance_f32` for the real valued input FFT. This structure should be allocated as a static variable so that it can be reused for all subsequent FFT calls.

The recommended way to initialise this structure is to use the dedicated CMSIS DSP library function that corresponds to the required FFT size. For example, to initialize a 256-point complex valued input FFT, you would call `arm_cfft_init_256_f32()` and to initialize a real 512-point real valued input FFT call: `arm_rfft_init_512_f32()`. The CMSIS DSP Library provides such initialization functions for FFT sizes ranging from 16 to 8192 points, and supports for 16-bit and 32-bit fixed-point and as well as 32-bit and 64 bit floating-point data types. To access these initialization functions in your C Code, include `transform_functions.h` in addition to `arm_math.h`.

◆ `arm_cfft_init_256_f32()`

`arm_status` `arm_cfft_init_256_f32 ( arm_cfft_instance_f32 * S )`

Initialization function for the cfft f32 function with 256 samples.

Parameters

[in, out] `S` points to an instance of the floating-point CFFT structure

Returns

execution status

- `ARM_MATH_SUCCESS` : Operation successful
- `ARM_MATH_ARGUMENT_ERROR` : an error is detected

Use of this function is mandatory only for the Helium and Neon versions of the FFT.

Other versions can still initialize directly the data structure using variables declared in `arm_const_structs.h`

Figure 4: Instance structure initialization function for floating-point CFFT [1].

2.2. FFT Preparation and Function Call

2.2.1. Type Cast from ADC Dual Mode Samples

When the ADC is operated in dual mode, both converted data from ADC1 and ADC2 are packed into a 32-bit register in this way:

```
ADC_CDR[31:0] = ADC2_DR[15:0] | ADC1_DR[15:0]
```

The samples must now first be separated, and can be cast as float at the same time. If you configure the dma suitably, you will get just as many values as you need for the FFT.

Accordingly you can cast the values in a for loop and copy them into the inout buffer for the fft.

```
...
float32_t sample_adc1;
float32_t sample_adc2;
for (n = 0; n < meas_samples_n; n++)
{
    sample_adc1 = (float32_t)(adc_dual_mode_samples[n] & 0x0000FFFF);
    sample_adc2 = (float32_t)((adc_dual_mode_samples[n] >> 16) & 0x0000FFFF);
    // todo : fill into the inout buffer for the FFT
    // format : { real[0], imag[0], real[1], imag[1], real[2], imag[2] ... }
}
```

Depending on which ADC channel you take as real or imaginary part, you have to switch or adjust a sign, so that in the end the sign of the frequency matches the expectations.

2.2.2. Optional Zero Padding

If you have fewer ADC samples than the selected FFT size, it is good practice to fill the remaining entries with zero padding.

2.2.3. FFT Function Call

The Complex FFT function call will look something like this:

```
arm_cfft_f32(&cfft_instance, cfft_inout, IFFT_FLAG, DO_BIT_REVERSE);
```

When the function is called, the array `cfft_inout` contains the complex time signal, which is then overwritten with the complex spectral values by the inplace calculation.

C pointer refresher:

```
float32_t cfft_inout[2*FFT_SIZE]
```

Since `cfft_inout` is an array, `cfft_inout` is interpreted as a pointer and points to the first element in the array. The same result you get if you write `&cfft_inout[0]`.

If you use the Real FFT function which is optimized for real input signals, then the call will look something like this:

```
arm_rfft_fast_f32(&rfft_instance, fft_in, fft_out, IFFT_FLAG);
```

`fft_in` and `fft_out` are arrays with the length `FFT_SIZE`.

When the function is called, the array `fft_in` contains the “real” time signal, and in the `fft_out` array the calculated complex spectral values will be stored.

Only `FFT_SIZE/2` complex spectral values will be stored in the `fft_out` buffer, since the second half of the data would be equals the conjugate of the first half flipped in frequency.

That's why the `fft_out` buffer does not need to be bigger. Attention the `fft_in` buffer will be manipulated by the FFT function and will not hold the time signal anymore after the function call.

2.2.4. Test Signal

You can easily check the FFT function by means of test signals even before you have put the ADC into operation.

The test signal can be e.g. a DC value (all 1s) or a sine or a complex signal like $e^{j\phi} = \cos(\phi) + j\sin(\phi)$.

You can either calculate the test signal in a function and fill it into the array or alternatively calculate it in Matlab, save it as CSV file and include it with #include.

2.3. Some useful CMSIS Functions

Besides the FFT function, CMSIS has some useful “array” functions that are also implemented very efficiently.

Some of them are e.g.

```
arm_cmplx_mag_f32  
arm_scale_f32  
arm_add_f32  
arm_max_f32
```

2.4. Performance Testing

If you want to compare the execution time of a function, you can, for example, set an available GPIO pin of the microcontroller to logic level **1** immediately before the regarded function call, and clear it to **0** afterwards.

The duration can then be measured either with an oscilloscope or by reading and comparing timer counter values.

2.5. Some possible errors

2.5.1. __FPU_PRESENT

```
../Drivers/CMSIS/Include/core_cm4.h:105:8: error: #error "Compiler generates FPU instructions for  
a device without an FPU (check __FPU_PRESENT)"  
#error "Compiler generates FPU instructions for a device without an FPU (check __FPU_PRESENT)"  
^~~~~
```

Possible fix:

- ⇒ include "stm32f4xx_hal.h" before you include "arm_math.h"
- ⇒ see also <https://community.st.com/s/question/0D50X0000At139aSQA/using-dsp-libraries-eg-armmathh-in-stm32cubeide-stm32f4>

2.5.2. Hard Fault when calling `arm_cfft_f32`

For the complex FFT:

Check that your inout buffer size is twice the fft size (array of complex numbers,
`cfft_inout[2*FFT_SIZE]`)

References

- [1] CMSIS DSP Software Library, [Link](#) (2026).
- [2] ST Applicatiuon Note AN4044, “Floating point unit demonstration on STM32 microcontrollers”, DocID022737 Rev 2, May 2016. [Link](#) (2025).
- [3] Th. Lorensen, «The DSP capabilities of ARM® Cortex®-M4 and Cortex-M7 Processors, DSP feature set and benchmarks», ARM, White Paper, November 2016. [Link](#) (2025).